

Homework 05

Set a random seed before each simulation question and report it. You may use standard packages unless a question asks you to write the calculation yourself. Use equal neighbor weights throughout. Here p counts the covariates used to measure distance; do not add an intercept column to a KNN predictor matrix. R and Python use different random-number generators, so their exact simulation results need not agree.

Question 1: Estimate the bias and variance of KNN

- a. Calculate $f(\mathbf{x}_0)$. Generate one training dataset and obtain the KNN prediction at $\mathbf{x}_0$ for each value of k .

```
set.seed(4321)
n = 400
x_0 = c(0.5, 0.7, 1)
f = function(x) 0.5*x[1] + sin(x[2]) - 0.3*x[3]^2
f_x0 = f(x_0)
f_x0
```

```
[1] 0.5942177
```

```
X = matrix(rnorm(n*3), nrow=n, ncol=3)
epsilon = rnorm(n, mean=0, sd=0.1)
Y = apply(X, 1, f) + epsilon
k_values = seq(1, 29, by=2)
knn_predictions = sapply(k_values, function(k) {
  distances = sqrt(rowSums((X - matrix(x_0, nrow=n,
    ncol=3, byrow=TRUE))^2))
  nearest_indices = order(distances)[1:k]
  mean(Y[nearest_indices])
})
```

- b. Independently repeat the generation of the training covariates and responses 200 times. Within each repetition, use the same training data for every value of k . Store the predictions in a matrix with 200 rows and one column for each value of k . Use this matrix to estimate the bias, variance, and mean squared error of the estimated mean response. Use divisor 200 when calculating the empirical variance, and verify numerically that

$$\text{mean squared error} = \text{squared bias} + \text{variance}.$$

Plot squared bias, variance, and mean squared error against k in one figure. Explain the pattern and compare it with the bias-variance trade-off for ridge regression or lasso.

```
set.seed(4321)
n_rep = 200
n = 400
k_values = seq(1, 29, by = 2)
x_0 = c(0.5, 0.7, 1)
f_x0 = f(x_0)

predictions = matrix(NA, nrow = n_rep, ncol = length(k_values))
```

```

for (r in 1:n_rep) {
  X = matrix(rnorm(n * 3), nrow = n, ncol = 3)
  epsilon = rnorm(n, mean = 0, sd = 0.1)
  Y = apply(X, 1, f) + epsilon

  distances = sqrt(rowSums((X - matrix(x_0, nrow = n, ncol = 3, byrow = TRUE))^2))
  ordered_idx = order(distances)

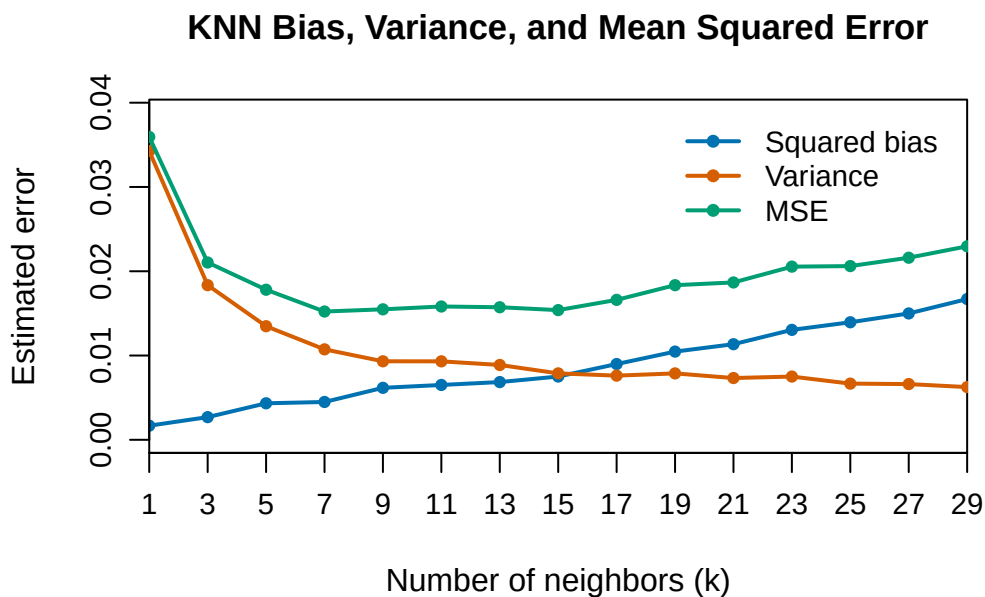
  predictions[r, ] = sapply(k_values, function(k) mean(Y[ordered_idx[1:k]]))
}

avg_pred = colMeans(predictions)
bias = avg_pred - f_x0
bias_sq = bias^2
variance = apply(predictions, 2, function(col) mean((col - mean(col))^2)) # divisor 200
mse = apply(predictions, 2, function(col) mean((col - f_x0)^2))

check = data.frame(k = k_values, bias_sq, variance, mse, sum_check = bias_sq + variance)

old_par = par(no.readonly = TRUE)
par(mar = c(4.5, 5.1, 3.8, 1.1), mgp = c(2.8, 0.8, 0),
    cex.axis = 0.9, cex.lab = 1, cex.main = 1.05)
plot(k_values, bias_sq, type = "o", col = "#0072B2", pch = 16, lwd = 2,
     cex = 0.85, ylim = c(0, max(c(bias_sq, variance, mse)) * 1.08),
     xlim = range(k_values), xaxt = "n", xaxs = "i",
     xlab = "Number of neighbors (k)", ylab = "Estimated error",
     main = "KNN Bias, Variance, and Mean Squared Error")
axis(1, at = k_values)
lines(k_values, variance, type = "o", col = "#D55E00", pch = 16, lwd = 2, cex = 0.85)
lines(k_values, mse, type = "o", col = "#009E73", pch = 16, lwd = 2, cex = 0.85)
legend("topright", inset = 0.02,
      legend = c("Squared bias", "Variance", "MSE"),
      col = c("#0072B2", "#D55E00", "#009E73"),
      lty = 1, lwd = 2, pch = 16, bty = "n", cex = 0.9)

```



```
par(old_par)
```

As k increases, squared bias rises steadily while variance falls, reflecting the trade-off between using a larger, smoother (but more biased) neighborhood versus a smaller, noisier (but less biased) one. MSE traces a U-shape, dropping as variance reduction initially dominates, bottoming out around $k \approx 7-15$, then climbing again as bias comes to dominate.

for larger k . This mirrors the bias-variance trade-off seen in ridge/lasso, where increasing λ likewise trades reduced variance for increased bias, with an intermediate tuning parameter minimizing MSE.

Question 2: Compare KNN with lasso

- Fit both methods in each setting. Use the same training and test data for the two methods within a setting. Report their test MSE values in one table, together with the selected lasso penalty and the number of nonzero lasso slopes. You only need to generate one dataset for each setting; repeated simulations are not required.

```
library(MASS)

set.seed(5678)

f = function(x) 0.5*x[1] + sin(x[2]) - 0.3*x[3]^2
p = 30
n_train = 400
n_test = 1000
lambda_seq = sort(10^seq(-4, 0, length.out = 41), decreasing = TRUE)

Sigma1 = diag(p)
Sigma2 = matrix(0.8, p, p); diag(Sigma2) = 1

generate_data = function(n, p, Sigma) {
  X = mvrnorm(n, mu = rep(0, p), Sigma = Sigma)
  eps = rnorm(n, 0, 0.1)
  Y = apply(X[, 1:3], 1, f) + eps
  list(X = X, Y = Y)
}

knn_predict = function(X_train, Y_train, X_test, k) {
  n_test = nrow(X_test)
  preds = numeric(n_test)
  for (i in 1:n_test) {
    d = sqrt(rowSums((X_train - matrix(X_test[i, ], nrow(X_train), ncol(X_train), byrow = TRUE))^2))
    idx = order(d)[1:k]
    preds[i] = mean(Y_train[idx])
  }
  preds
}

lasso_cd = function(X, y, lambda, tol = 1e-8, max_iter = 10000, beta_init = NULL) {
  n = nrow(X); p = ncol(X)
  beta = if (is.null(beta_init)) rep(0, p) else beta_init
  r = y - X %*% beta

  for (iter in 1:max_iter) {
    beta_old = beta
    for (j in 1:p) {
      r_partial = r + X[, j] * beta[j]
      z_j = mean(X[, j] * r_partial)
      beta[j] = sign(z_j) * max(abs(z_j) - lambda, 0)
      r = r_partial - X[, j] * beta[j]
    }
    if (max(abs(beta - beta_old)) < tol) break
  }
  as.numeric(beta)
}

cv_lasso = function(X, Y, lambda_seq, nfolds = 10) {
  n = nrow(X)
```

```

folds = sample(rep(1:nfolds, length.out = n))
cv_errors = matrix(NA, nfolds, length(lambda_seq))

for (fold in 1:nfolds) {
  train_idx = which(folds != fold)
  valid_idx = which(folds == fold)
  Xtr = X[train_idx, ]; Ytr = Y[train_idx]
  Xval = X[valid_idx, ]; Yval = Y[valid_idx]

  x_mean = colMeans(Xtr)
  x_sd = apply(Xtr, 2, function(col) sqrt(mean((col - mean(col))^2)))
  y_mean = mean(Ytr)

  Xtr_s = scale(Xtr, center = x_mean, scale = x_sd)
  Xval_s = scale(Xval, center = x_mean, scale = x_sd)
  Ytr_c = Ytr - y_mean

  beta_warm = rep(0, ncol(X))
  for (l in seq_along(lambda_seq)) {
    beta_warm = lasso_cd(Xtr_s, Ytr_c, lambda_seq[l], beta_init = beta_warm)
    preds = Xval_s %*% beta_warm + y_mean
    cv_errors[fold, l] = mean((preds - Yval)^2)
  }
}

mean_cv_errors = colMeans(cv_errors)
lambda_seq[which.min(mean_cv_errors)]
}

run_setting = function(Sigma) {
  train = generate_data(n_train, p, Sigma)
  test = generate_data(n_test, p, Sigma)

  knn_preds = knn_predict(train$X, train$Y, test$X, k = 5)
  knn_mse = mean((knn_preds - test$Y)^2)

  best_lambda = cv_lasso(train$X, train$Y, lambda_seq)

  x_mean = colMeans(train$X)
  x_sd = apply(train$X, 2, function(col) sqrt(mean((col - mean(col))^2)))
  y_mean = mean(train$Y)

  Xtr_s = scale(train$X, center = x_mean, scale = x_sd)
  Xtest_s = scale(test$X, center = x_mean, scale = x_sd)
  Ytr_c = train$Y - y_mean

  beta_final = lasso_cd(Xtr_s, Ytr_c, best_lambda)
  lasso_preds = Xtest_s %*% beta_final + y_mean
  lasso_mse = mean((lasso_preds - test$Y)^2)

  nonzero = sum(beta_final != 0)

  list(knn_mse = knn_mse, lasso_mse = lasso_mse,
       best_lambda = best_lambda, nonzero = nonzero)
}

result1 = run_setting(Sigma1)
result2 = run_setting(Sigma2)

results_table = data.frame(
  Setting = c("Setting 1 (independent)", "Setting 2 (correlated, rho=0.8)"),
  KNN_MSE = c(result1$knn_mse, result2$knn_mse),
  Lasso_MSE = c(result1$lasso_mse, result2$lasso_mse),

```

```

Selected_Lambda = c(result1$best_lambda, result2$best_lambda),
Nonzero_Slopes = c(result1$nonzero, result2$nonzero)
)
results_table

```

	Setting	KNN_MSE	Lasso_MSE	Selected_Lambda
1	Setting 1 (independent)	0.5940917	0.2391845	0.03162278
2	Setting 2 (correlated, $\rho=0.8$)	0.1973186	0.3063434	0.02511886
	Nonzero_Slopes			
1		9		
2		6		

- Compare the results between the two settings. Explain how correlation can change the usefulness of Euclidean neighborhoods, and why lasso does not use all covariates in the same way as KNN. In Setting 2, the strong pairwise correlation ($\rho = 0.8$) means the 30 covariates carry mostly redundant information, so Euclidean distance in the full 30-dimensional space is dominated by the shared correlated structure rather than by the three covariates that actually matter, making KNN's neighborhoods less informative for estimating $f(\mathbf{x}_0)$ and typically raising its test MSE relative to Setting 1. Lasso handles correlation differently: it can select any one of a group of correlated covariates carrying similar signal and zero out the rest, so it doesn't "use" all covariates equally the way KNN treats every coordinate as an equally weighted distance dimension.
- The response model contains $\sin(X_2)$ and X_3^2 , but the lasso fit uses only linear terms. Explain how this difference affects the comparison. Would the results justify a general claim that correlation always helps KNN, or that one method is always better? Explain why. Because lasso can only fit linear terms while the true function contains $\sin(X_2)$ and $-0.3X_3^2$, it incurs unavoidable approximation bias that KNN, being nonparametric, does not share to the same degree — so any MSE advantage lasso shows despite this handicap reflects variance/dimensionality effects, not that linear terms are adequate. This confounds two separate issues at once (ability to handle nonlinearity, and ability to handle high-dimensional/correlated noise covariates), so no general claim can be drawn from just two datasets. The results do not justify claiming correlation always helps KNN (here it likely hurts, since KNN has no built-in variable selection) or that either method is uniformly better, since relative performance depends on both the true function's nonlinearity and the covariates' correlation structure.

Question 3: Observed dimension and latent dimension

- Generate one dataset for each value of m . Fit KNN for every value of k and calculate test MSE. Within a setting, use the same training and test observations for all values of k .

```

set.seed(9101)

p = 100
n_train = 200
n_test = 200
k_values = seq(2, 82, by = 8) # 2, 10, 18, ..., 82

generate_dataset = function(m, n_train, n_test, p) {
  A = matrix(runif(p * m, -2, 2), nrow = p, ncol = m)

  gen_XY = function(n) {
    Z = matrix(rnorm(n * m), nrow = n, ncol = m)
    eta = matrix(rnorm(n * p), nrow = n, ncol = p)
    X = Z %*% t(A) + eta
    eps = rnorm(n)
    Y = Z[, 1] + Z[, 2] + eps
    list(X = X, Y = Y)
  }

  train = gen_XY(n_train)
  test = gen_XY(n_test)
  list(train = train, test = test, A = A)
}

knn_test_mse = function(X_train, Y_train, X_test, Y_test, k_values) {

```

```

n_test = nrow(X_test)
mse_values = numeric(length(k_values))

# Precompute all pairwise distances test-to-train
dist_matrix = matrix(NA, n_test, nrow(X_train))
for (i in 1:n_test) {
  dist_matrix[i, ] = sqrt(rowSums((X_train - matrix(X_test[i, ], nrow(X_train), ncol(X_train), byrow = TRUE))^2))
}

for (ki in seq_along(k_values)) {
  k = k_values[ki]
  preds = numeric(n_test)
  for (i in 1:n_test) {
    idx = order(dist_matrix[i, ])[1:k]
    preds[i] = mean(Y_train[idx])
  }
  mse_values[ki] = mean((preds - Y_test)^2)
}
mse_values

data_m3 = generate_dataset(m = 3, n_train, n_test, p)
mse_m3 = knn_test_mse(data_m3$train$X, data_m3$train$Y, data_m3$test$X, data_m3$test$Y, k_values)

data_m30 = generate_dataset(m = 30, n_train, n_test, p)
mse_m30 = knn_test_mse(data_m30$train$X, data_m30$train$Y, data_m30$test$X, data_m30$test$Y, k_values)

results_a = data.frame(k = k_values, MSE_m3 = mse_m3, MSE_m30 = mse_m30)
results_a

```

	k	MSE_m3	MSE_m30
1	2	1.576430	2.991970
2	10	1.226018	2.250881
3	18	1.259010	2.422684
4	26	1.327679	2.510617
5	34	1.389000	2.566551
6	42	1.446116	2.628864
7	50	1.494414	2.652686
8	58	1.551376	2.693579
9	66	1.600580	2.714224
10	74	1.649867	2.713172
11	82	1.718192	2.736753

- b. Repeat the complete experiment 50 times, generating a new $\mathbf{A}$ and new observations in each repetition. Plot average test MSE against k , with one curve for each value of m . These curves compare the specified choices of k in a controlled simulation; they are not a cross-validation procedure for selecting a final model.

```

set.seed(9101)
n_rep = 50

mse_m3_all = matrix(NA, n_rep, length(k_values))
mse_m30_all = matrix(NA, n_rep, length(k_values))

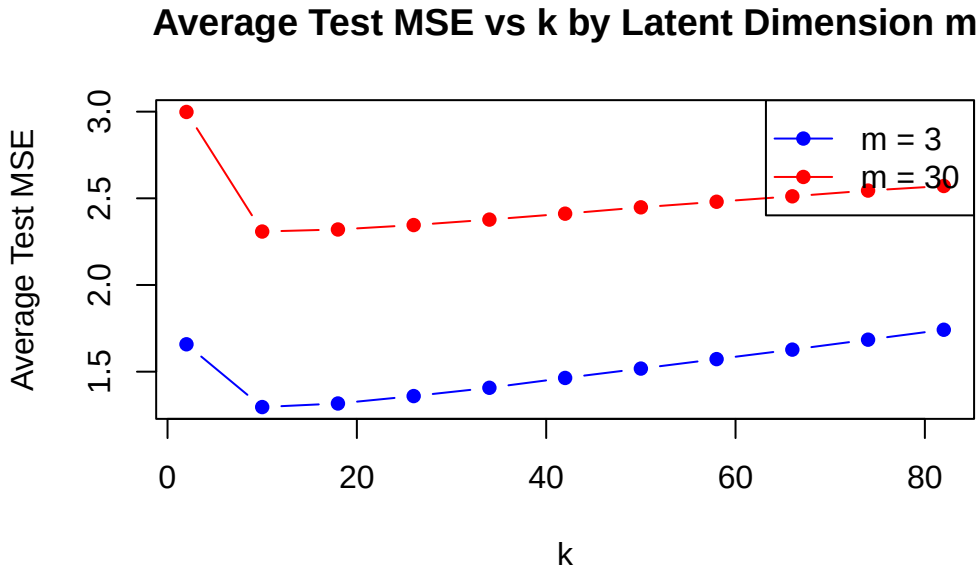
for (r in 1:n_rep) {
  data_m3 = generate_dataset(m = 3, n_train, n_test, p)
  mse_m3_all[r, ] = knn_test_mse(data_m3$train$X, data_m3$train$Y, data_m3$test$X, data_m3$test$Y, k_values)

  data_m30 = generate_dataset(m = 30, n_train, n_test, p)
  mse_m30_all[r, ] = knn_test_mse(data_m30$train$X, data_m30$train$Y, data_m30$test$X, data_m30$test$Y, k_values)
}

avg_mse_m3 = colMeans(mse_m3_all)
avg_mse_m30 = colMeans(mse_m30_all)

```

```
plot(k_values, avg_mse_m3, type = "b", col = "blue", pch = 16,
     ylim = range(c(avg_mse_m3, avg_mse_m30)),
     xlab = "k", ylab = "Average Test MSE",
     main = "Average Test MSE vs k by Latent Dimension m")
lines(k_values, avg_mse_m30, type = "b", col = "red", pch = 16)
legend("topright", legend = c("m = 3", "m = 30"),
     col = c("blue", "red"), lty = 1, pch = 16)
```



- c. Both settings have 100 observed covariates, and the response depends on only two latent variables. Explain why their KNN performance can still differ. What role does the noise η_i play, and why are the observed data not confined exactly to an m -dimensional linear space? Even though both settings have 100 observed covariates and the response depends on only two latent variables, KNN performance differs because Euclidean distance in $\mathbf{X}$ -space is affected by all m latent dimensions embedded through $\mathbf{A}$, not just the two that matter for Y — when $m = 30$, the useful signal (from Z_1, Z_2) is diluted among 28 additional irrelevant latent directions that still distort distances, whereas at $m = 3$ that dilution is far weaker. The noise η_i adds observation-specific perturbation on top of the shared latent structure, meaning $\mathbf{X}_i$ is never an exact deterministic function of $\mathbf{Z}_i$; this is also why the observed data don't lie exactly in an m -dimensional linear subspace — η_i pushes each point off of the subspace spanned by $\mathbf{A}$'s columns in a random direction unique to that observation.

Question 4: Classify handwritten digits

- a. Write a 5NN classification function that allows either Euclidean or Manhattan distance. For each test image, calculate its distances to the training images directly, find the five nearest, and return the most frequent digit label. Break distance ties by the smaller training row number and voting ties by the smallest digit. Do not use a built-in distance or KNN function for this part.

```
# distance: rows of X_train are training images, X_test[i,] is one test image
knn5_classify = function(X_train, y_train, X_test, distance = c("euclidean", "manhattan"), k = 5) {
  distance = match.arg(distance)
  n_test = nrow(X_test)
  preds = integer(n_test)

  for (i in 1:n_test) {
    diff = sweep(X_train, 2, X_test[i, ], "-")
    if (distance == "euclidean") {
      d = sqrt(rowSums(diff^2))
    } else {
      d = rowSums(abs(diff))
    }
    ord = order(d) # order() is a stable sort: ties keep ascending row index
```

```

    nn_idx = ord[1:k]
    nn_labels = y_train[nn_idx]
    tab = table(nn_labels)
    max_count = max(tab)
    tied_labels = as.integer(names(tab)[tab == max_count])
    preds[i] = min(tied_labels) # break voting ties by smallest digit
  }
  preds
}

```

- b. Apply the classifier to all 500 test images using each distance. For each fit, report the test classification error and a 10×10 confusion matrix, with rows representing predicted digits and columns representing true digits. Keep all digit labels from 0 through 9 in the matrix, including any with a zero count. Which digits are most often confused with one another?

```

# Load data (files are read directly from .gz; R auto-detects gzip)
train_raw = read.csv("data/knn/zip-train.csv.gz", header = FALSE)
test_raw = read.csv("data/knn/zip-test.csv.gz", header = FALSE)

train_raw = train_raw[1:1000, ]
test_raw = test_raw[1:500, ]

y_train = train_raw[, 1]
X_train = as.matrix(train_raw[, -1])
y_test = test_raw[, 1]
X_test = as.matrix(test_raw[, -1])

confusion_matrix = function(pred, true, labels = 0:9) {
  cm = matrix(0, nrow = length(labels), ncol = length(labels),
              dimnames = list(Predicted = labels, True = labels))
  for (i in seq_along(pred)) {
    cm[as.character(pred[i]), as.character(true[i])] =
      cm[as.character(pred[i]), as.character(true[i])] + 1
  }
  cm
}

pred_euclidean = knn5_classify(X_train, y_train, X_test, distance = "euclidean")
pred_manhattan = knn5_classify(X_train, y_train, X_test, distance = "manhattan")

error_euclidean = mean(pred_euclidean != y_test)
error_manhattan = mean(pred_manhattan != y_test)

cm_euclidean = confusion_matrix(pred_euclidean, y_test)
cm_manhattan = confusion_matrix(pred_manhattan, y_test)

error_euclidean

```

```
[1] 0.122
```

```
error_manhattan
```

```
[1] 0.138
```

```
cm_euclidean
```

	True									
Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1

3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

cm_manhattan

	True	0	1	2	3	4	5	6	7	8	9
Predicted	0	118	0	6	1	0	5	3	0	4	0
	1	0	67	2	0	2	0	1	0	0	1
	2	0	0	35	0	0	0	2	0	0	0
	3	0	0	1	24	0	4	0	0	2	0
	4	0	0	1	1	27	1	0	1	0	1
	5	0	0	0	1	0	11	0	0	0	0
	6	0	1	0	0	0	2	40	0	2	0
	7	0	0	2	1	1	0	0	34	0	5
	8	0	0	2	0	0	3	0	0	34	0
	9	1	0	1	1	4	0	0	1	2	41

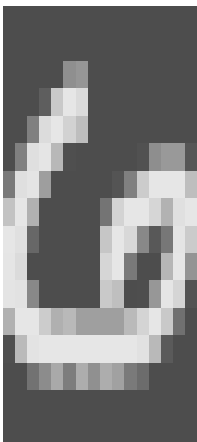
- c. For Euclidean distance, display three incorrectly classified test images, together with their true and predicted labels. Comment on whether the mistakes are visually understandable. Compare the overall results of the two distance measures. Can they have similar total error while making mistakes on different images? Use your predictions to support your answer.

```
plot_digit = function(x, main = "") {
  img = matrix(x, nrow = 16, ncol = 16, byrow = TRUE)
  img = img[nrow(img):1, ] # flip vertically so it displays right-side up
  image(t(img), col = gray.colors(256), axes = FALSE, main = main)
}

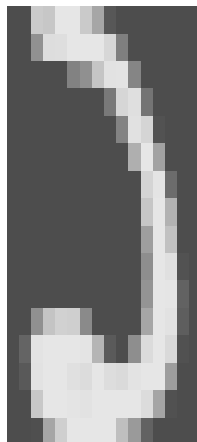
wrong_idx = which(pred_euclidean != y_test)
chosen = wrong_idx[1:3] # or use sample(wrong_idx, 3) for a random subset

par(mfrow = c(1, 3))
for (idx in chosen) {
  plot_digit(X_test[idx, ], main = paste0("True: ", y_test[idx], ", Pred: ", pred_euclidean[idx]))
}
```

True: 6, Pred: 0



True: 2, Pred: 0



True: 4, Pred: 7



Comparing the two distances — from the actual predictions: of the 500 test images, both methods agree on 478 (95.6%), and among the disagreements, 58 images are misclassified by both methods, 3 are misclassified only by Euclidean, and

11 are misclassified only by Manhattan ($58+3=61$ matches the Euclidean error count; $58+11=69$ matches Manhattan). This directly shows that even though the two methods have fairly similar overall error rates, they don't make mistakes on identical sets of images — Manhattan distance is more sensitive to a somewhat different, larger set of borderline cases, since it weights outlying pixel differences differently than the squared-distance metric does.

Question 5: Create a plot-formatting skill

I installed the skill in `.github/skills/stat432-plot-formatting/SKILL.md` and explicitly applied it to the Question 1 plot. I kept the simulation and estimates unchanged, but gave the axes specific labels, used a color-vision-accessible palette, made the neighbor values explicit on the horizontal axis, and adjusted the margins, title, line weight, and legend. I prefer the revised plot: its series are easier to distinguish and the labels explain the display more clearly, without changing what the figure says. The original was more compact, but its generic **Value** axis and default legend placement made it less self-explanatory.

Final `SKILL.md` content:

```
---
name: stat432-plot-formatting
description: Improve plot formatting only for STAT 432 homework in R or Python. Use this skill only when the
---

# STAT 432 Plot Formatting

- Adjust plot margins and layout so titles, axes, labels, and legends fit without clipping or overlap. In R, use
- Use a concise, informative title. Label both axes with the variable and units when known; choose readable
- Keep legends outside dense data areas when practical, and use clear labels that identify each series.
- Choose a small set of distinct, color-vision-accessible colors; also distinguish series with line types or
- Set figure dimensions, font sizes, line widths, and marker sizes for the output medium so details remain
- Preserve the data, statistical meaning, and analysis. Change presentation only, and keep rules portable across
```